MPI Collectives on 2D/3D Mesh Topologies: Implementation, Benchmarking, Chunking, and Visualization

Authors:
Aryan Maskara (2023111004)
Harshil Singh (2023111003)
Kushal Balabhadruni (2023101039)

November 30, 2025

Contents

1 Executive Summary **3**

2 Repository Scope and Artifacts **3**

3 Algorithms Implemented **3**
3.1 2D Broadcast and 2D Reduce 3
3.2 3D Broadcast and 3D Reduce 4
3.3 Chunking Experiments 4

4 Exact Experimental Workflow (What Was Used and How) **4**
4.1 Build and Run Pipeline 4
4.2 Process Counts and Mesh Sizes 4
4.3 Timing Data Collection 5
4.4 System Jitter Considerations 5

5 Plot and Analysis Pipeline **5**
5.1 Main Performance Plots 5
5.2 Chunking Plots and Summary 6

6 Measured Results and Improvements **6**
6.1 Main Collective Performance (Largest Process Count) 6
6.2 Chunking Improvements (4 MB Message, 64 Processes) 6
6.3 Data Coverage Note 7

7 Figures from Repository Outputs **7**

8 Interactive Visualizations **8**

9 Reproducibility Checklist **8**

10 Conclusion **9**

1 Executive Summary

This repository implements topology-aware MPI collective communication algorithms for 2D and 3D mesh process layouts, evaluates them against brute-force baselines, and adds large-message chunking experiments. The project includes:

- C++/MPI implementations for `Broadcast` and `Reduce` in both 2D and 3D.
- End-to-end run scripts to build and execute all experiments.
- Timing datasets, plotting scripts, generated plots, and chunking summaries.
- Interactive HTML visualizations for algorithm behavior.

This report documents **what was used**, **how it was used**, **what data was collected**, and **what improvements were observed**.

2 Repository Scope and Artifacts

Top-level artifacts used in this report:

- Implementations: `2D/`, `3D/`, and `Chunking/`.
- Timing files: `2D-Broadcast-times.txt`, `2D-Reduce-times.txt`, `3D-Broadcast-times.txt`, `3D-Reduce-times.txt`.
- Chunking dataset: `Chunking/chunking_times.txt`.
- Plot generators: `visualize_performance.py` and `Chunking/visualize_chunk_variation.py`.
- Output plots: `plots/` and `chunking_plots/`.
- Orchestration script: `run_all.sh`.
- Visual simulation pages: `website-visualizations/`.

3 Algorithms Implemented

3.1 2D Broadcast and 2D Reduce

Executables in both `2D/Broadcast/` and `2D/Reduce/`:

- `brute`: root sends/receives directly to/from all nodes.
- `bfs`: wavefront propagation using neighborhood fanout.
- `recursive`: quadrant-style recursive dissemination/aggregation.
- `row_pipeline`: dimension-order pipeline (row-then-column or reverse for reduce).
- `sqrt`: hierarchical block-based method.

3.2 3D Broadcast and 3D Reduce

Executables in both `3D/Broadcast/` and `3D/Reduce/`:

- `brute`
- `bfs`
- `recursive`
- `row_pipeline` (dimension-order in 3 axes)
- `sqrt` (hierarchical/block-style)

3.3 Chunking Experiments

In `Chunking/2D/` and `Chunking/3D/`:

- `naive_unserialized`: sends full message directly.
- `chunked_serialized`: splits message and pipelines chunks.

4 Exact Experimental Workflow (What Was Used and How)

4.1 Build and Run Pipeline

The main entry point is:

```
chmod +x run_all.sh
./run_all.sh
```

The script does the following in order:

- Builds all MPI binaries in `2D/Broadcast`, `2D/Reduce`, `3D/Broadcast`, `3D/Reduce`, `Chunking/2D`, and `Chunking/3D`.
- Runs all experiment scripts:
  - `2D/Broadcast/script.sh`
  - `2D/Reduce/script.sh`
  - `3D/Broadcast/script.sh`
  - `3D/Reduce/script.sh`
  - `Chunking/2D/script.sh`
  - `Chunking/3D/script.sh`
- Generates plots via `visualize_performance.py` and `Chunking/visualize_chunk_variation.py`.

4.2 Process Counts and Mesh Sizes

- 2D runs: `sizes=(16 81 169 256 400)` corresponding to 4×4, 9×9, 13×13, 16×16, and 20×20.
- 3D runs: `sizes=(27 64 216 343 729)` corresponding to 3^3, 4^3, 6^3, 7^3, and 9^3.
- Chunking runs: fixed `-np 64`, i.e., 8×8 (2D) and $4 \times 4 \times 4$ (3D).

4.3 Timing Data Collection

The experiment scripts print timings to stdout for each run and algorithm. Timing files are then maintained in repository text files used by plotting:

- `2D-Broadcast-times.txt`
- `2D-Reduce-times.txt`
- `3D-Broadcast-times.txt`
- `3D-Reduce-times.txt`

Each algorithm section stores three trials per processor count, e.g.:

```
AlgorithmName:
169: t1 t2 t3
256: t1 t2 t3
```

4.4 System Jitter Considerations

Measured MPI times include small run-to-run variation (system jitter) caused by OS scheduling, transient background load, memory/cache effects, and MPI runtime progress overhead. This is more visible for short runs and in oversubscribed settings (used in chunking scripts).

To reduce jitter impact, experiments are recorded with three trials per configuration and analysis is based on trial means. Runtime is reported from the slowest rank completion point (via max-time aggregation in code), which avoids underestimating collective completion time.

5 Plot and Analysis Pipeline

5.1 Main Performance Plots

`visualize_performance.py` supports both a single combined file and split timing files. In this repository, split files are used. The parser:

- Normalizes operation names (`Bcast` and `Broadcast`).
- Normalizes algorithm names (`row_pipeline` to `Dimension Order Pipeline`).
- Merges section data and computes trial means.

Generated outputs include:

- `plots/2d_broadcast_combined.png`
- `plots/2d_reduce_combined.png`
- `plots/3d_broadcast_combined.png`
- `plots/3d_reduce_combined.png`
- corresponding `_no_brute` versions.

5.2 Chunking Plots and Summary

`Chunking/visualize_chunk_variation.py` uses:

- Fixed message size: $2^{22} = 4$ MB.
- Chunk sizes: 1KB, 4KB, 16KB, 64KB, 256KB, 1MB, 2MB, 4MB.
- `naive_unserialized` baseline (stored as `naive` rows in `Chunking/chunking_times.txt`) averaged across all rows.

It generates comparison plots and `chunking_plots/chunk_variation_summary.txt` with best chunk markers.

6 Measured Results and Improvements

6.1 Main Collective Performance (Largest Process Count)

The table below summarizes average times (seconds) at the largest evaluated process count per case.

Table 1: Largest-scale averages and improvement over brute baseline

Case	Brute Avg (s)	Best Algorithm Avg (s)	Speedup
2D Broadcast (400 proc)	0.3255	Recursive: 0.0060	54.1x
2D Reduce (400 proc)	0.3499	SQRT: 0.0031	114.6x
3D Broadcast (729 proc)	0.0386	BFS: 0.0126	3.05x
3D Reduce (729 proc)	1.8688	Dim.-Order Pipeline: 0.0169	110.8x

Key observations:

- Topology-aware methods consistently reduce runtime compared to brute-force.
- 3D Reduce shows the largest gain due to severe brute-force root bottleneck.
- 2D Broadcast and 2D Reduce both show strong benefits from recursive/pipeline/hierarchical routing.

6.2 Chunking Improvements (4 MB Message, 64 Processes)

From `chunk_variation_summary.txt`:

- 2D `naive_unserialized` baseline: 0.045397 s. Best chunk size: 256KB at 0.018860 s (2.41x speedup).
- 3D `naive_unserialized` baseline: 0.037721 s. Best chunk size: 64KB at 0.016549 s (2.28x speedup).

Very small (1KB) and very large (4MB) chunk sizes underperform due to per-chunk overhead and reduced pipelining benefit.

6.3 Data Coverage Note

The current `2D-Broadcast-times.txt` has SQRT values for 16, 81, and 256 processes; entries for 169 and 400 are absent. All available measurements are still plotted and analyzed.

7 Figures from Repository Outputs

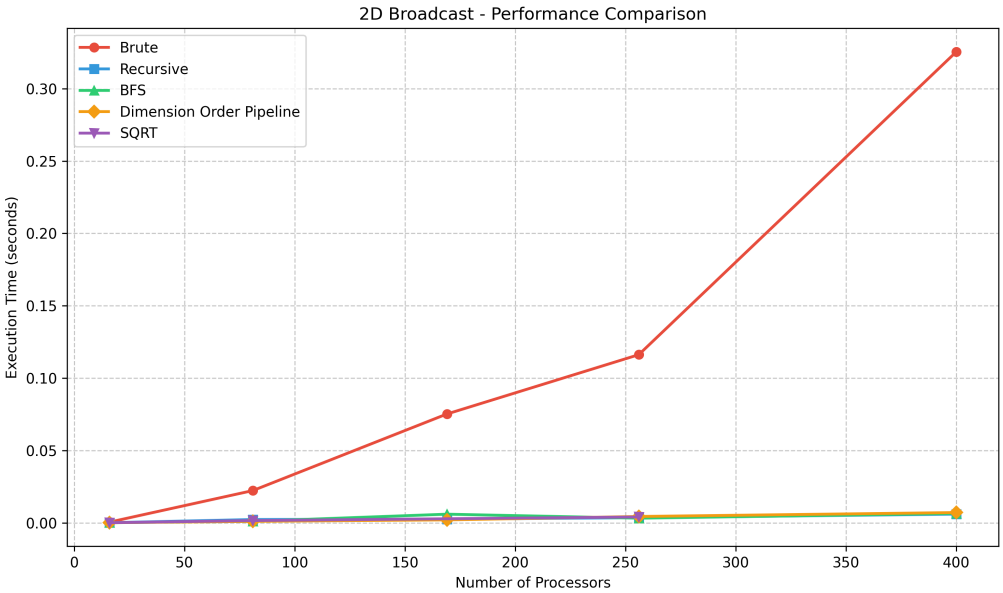

Figure 1: 2D Broadcast: Combined comparison

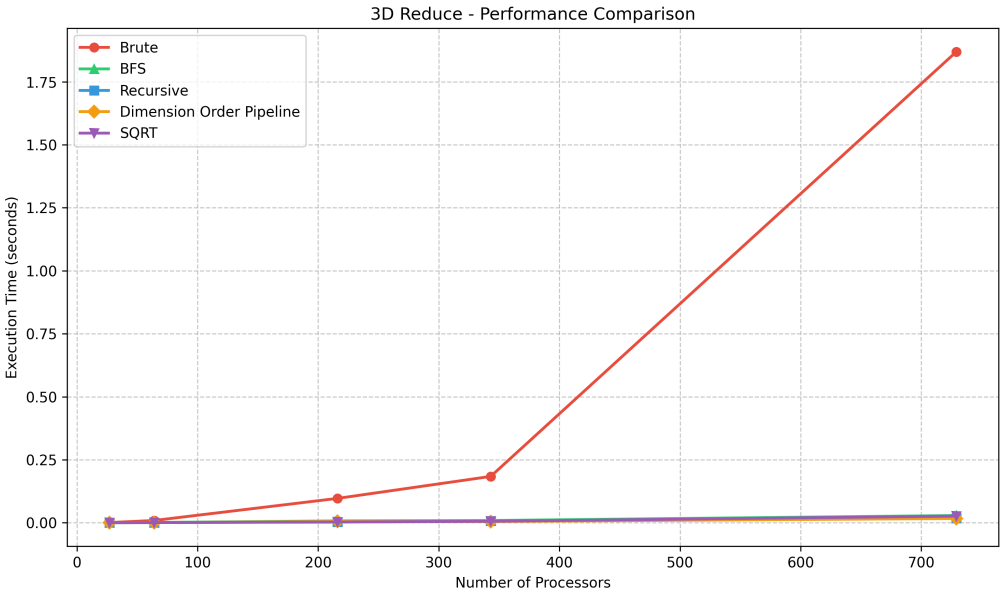

Figure 2: 3D Reduce: Combined comparison

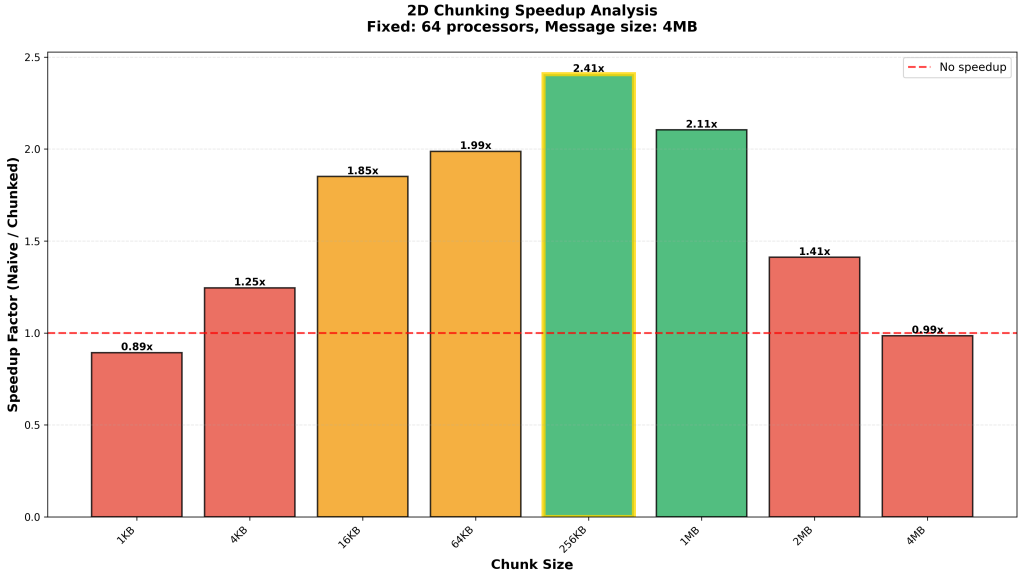

Figure 3: 2D Chunking speedup vs chunk size

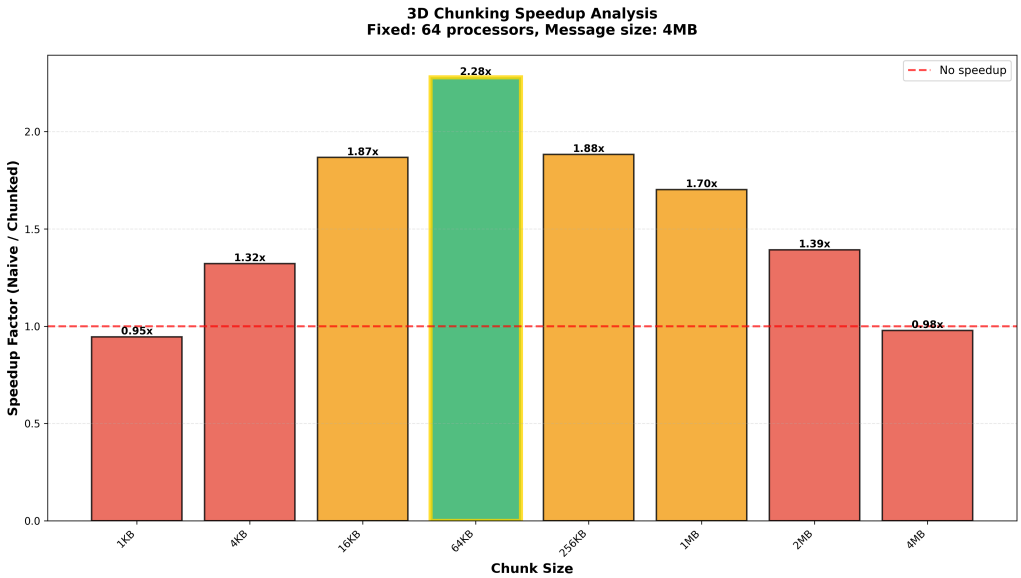

Figure 4: 3D Chunking speedup vs chunk size

8 Interactive Visualizations

The `website-visualizations/` folder provides browser-based interactive simulations for Broadcast and Reduce algorithm behavior. These pages are useful for validating communication patterns qualitatively (message flow, frontier expansion, and round progression) in addition to quantitative runtime plots.

9 Reproducibility Checklist

To reproduce all artifacts in this report:

- Build and run all experiments via `./run_all.sh`.

- Ensure timing files are populated with current measured data.
- Regenerate plots using:
  - `python3 visualize_performance.py`
  - `cd Chunking && python3 visualize_chunk_variation.py`
- Rebuild the PDF from `project_report.tex` after plot refresh.

10 Conclusion

This repository now contains a full implementation-and-evaluation pipeline: mesh-aware MPI collective algorithms, controlled benchmarks across process scales, chunking optimization for large messages, and both static/interactive visualization artifacts. The measured data confirms substantial gains over brute-force communication, especially for reduce operations at larger scales, while chunking provides additional 2.28x–2.41x improvements for 4 MB broadcasts at 64 processes when chunk sizes are tuned.